



Implementation of an AI-assisted contract review pipeline on Amazon Web Services (AWS)

Module Assignment for

CS5024 - Theory and Practice of Advanced AI Ecosystems

Student Name : **Shubham Yashwant Ghanmode**

Student ID : **25138928**

Revision Timestamp: 25/04/2026 03:21:42

Module Director: Patrick Denny
Teaching Assistant: Ankita Saini

Abstract

This project is an AWS-native prototype for contract review that ingests a PDF contract, extracts text, organizes clauses, and returns a semantic risk summary in plain English.

The problem was selected because contract review is repetitive, text-heavy, and well suited to managed cloud orchestration.

AWS was chosen because it is widely used in industry for scalable systems and provides a portfolio-ready way to combine event-driven processing, OCR, model inference, and durable state tracking.

The implementation uses API Gateway, Lambda, S3, Step Functions, Textract, Bedrock, DynamoDB, and CloudWatch to form a complete workflow.

The result is a decision-support prototype rather than legal advice, with the documented sample-contract run completing end to end and producing identifiable high- and medium-risk findings.

Contents

Abstract.....	2
Introduction	3
AI Ecosystem Architecture Used	3
Model Description	7
Scalability Considerations	10
References	12
Figure 1: Architecture used to create AI/ML ecosystem for Contract Review.....	4
Figure 2: Upload Curl request	4
Figure 3: S3 Event Notifications	5
Figure 4: CloudWatch Logs of the pipeline	5
Figure 5: DynamoDB Table	5
Figure 6: Step Function State Machine implementation graph	7
Figure 7: Successful Results API call	8
Figure 8: Files under the /processed/{jobid}/ directory	8
Table 1: Resources and their Purpose.....	3

Introduction

From the user side, the problem is simple. A person has a contract in PDF form and wants to know which parts might be commercially risky before taking it to a solicitor. They do not want to read the whole document line by line first, and they do not want a black-box answer with no explanation. They want something practical: upload the file, wait for processing, and receive a short summary plus a list of clauses that deserve attention.

I approached that user need as a cloud workflow rather than a single AI prompt. Functionally, the system had to accept an upload, track the job, extract text, split the contract into clauses, identify risky language, and return the result through an API. Non-functionally, it had to be asynchronous, easy to observe, and built from managed AWS services that could be demonstrated clearly in the assignment. That is why I separated the system into small responsibilities instead of trying to build everything inside one function.

The architecture also reflects the type of AI problem I was trying to solve. Contract review is not only about optical character recognition, and it is not only about classification. The system first has to turn a PDF into text, then turn text into clause-sized records, and only then can an AI model reason about meaning, asymmetry, liability, termination rights, indemnity, or data-use wording. I therefore used Amazon Textract for document extraction, a deterministic preprocessing stage in Lambda for auditability, and Amazon Bedrock for the semantic part of the task.

Another reason I chose this topic is that it shows why AWS is useful for AI systems beyond simple model calls. Upload handling, storage, OCR, orchestration, inference, API retrieval, and logging are all distinct concerns. AWS lets me connect those concerns with managed services instead of building the plumbing myself.

AI Ecosystem Architecture Used

I used a serverless AWS architecture in eu-west-1 with one S3 bucket and three public API routes. The main deployed resources are S3 bucket, the DynamoDB table, the Step Functions state machine, the shared Lambda role, and the Step Functions role.

Resource	Purpose
S3 bucket	Stores uploaded PDFs and processed outputs
DynamoDB table	Stores job status, summary fields, and audit metadata
State machine	Orchestrates Textract, preprocessing, and Bedrock
Lambda role	Shared execution role for project Lambdas
Step Functions role	Allows workflow execution and Lambda invocation
Bedrock model (Amazon Nova Pro)	AI inference model

Table 1: Resources and their Purpose

The API entry points are POST /upload-url, GET /status/{jobId}, and GET /results/{jobId}. The following Table 1 briefly explains the resources and their purposes



```
C:\Users\Shubham\Downloads>curl.exe -X POST "https://laex943sbc.execute-api.eu-west-1.amazonaws.com/upload-url" -H "Content-Type: application/json" -d '{"filename":"sample-contract.pdf"}'
```

Figure 2: Upload Curl request

The client then uploads the PDF directly to S3 instead of sending the whole file through Lambda. I chose that design because it is simpler, scales better for document uploads, and fits common AWS ingestion patterns.

Implementation of an AI-assisted contract review pipeline on Amazon Web Services (AWS)

Event notifications (1)						Edit	Delete	Create event notification
Send a notification when specific events occur in your bucket. Learn more								
☐	Name	Event types	Filters	Destination type	Destination			
☐	PipelineNotifEvent	All object create events	uploads/, .pdf	Lambda function	PipelineTrigger			

Figure 3: S3 Event Notifications

Once the file lands in S3, our event notification in S3 calls the PipelineTrigger Lambda, which starts the asynchronous processing path (Amazon Web Services, 2026). Refer to Figure 3 for the event notification setup. I added prefix and suffix filters for the uploads/ directory only, and for pdf files, so that the pipeline is not triggered for anything else.

We use the CloudWatch service to log the flow, to identify any potential issues and to track the flow status. Figure 4 shows AWS CloudWatch logs of the pipeline being triggered.

Timestamp	Message
2026-04-23T16:31:30.295Z	START RequestId: 36e9adf2-4766-4769-9aea-cfb90376cdea Version: \$LATEST
2026-04-23T16:31:30.296Z	Received S3 event: {"Records": [{"eventVersion": "2.1", "eventSource": "aws:s3", "awsRegion": "eu-west-1", "eventTime": "2026-04-23T16:31:29.199Z", "eventName": "ObjectCreated:Put", "userIdentity": {"principalId": "AWS:AROAZ7EKTHJK6ERIA4GEJL:UploadHandler"}, "requestParameters": {"sourceIPAddress": "104.28.161.205"}, "responseElements": {"x-amz-request-id": "3C8KXRY51RVRI7HQ", "x-amz-id-2": "RHeEjAAuYpQ3MB2o1BPX/4BN1Pp5EA8jC4vMgBjk3IKoEAoNR0b41NUFDtGMRzLAdj5AnhaEvo9A/+3VpoVXNeB08VU1Us"}, "s3": {"s3SchemaVersion": "1.0", "configurationId": "PipelineNotifEvent", "bucket": {"name": "contract-analyzer-bucket-shubham", "ownerIdentity": {"principalId": "A1MF77XR9Z5JQ2"}, "arn": "arn:aws:s3::contract-analyzer-bucket-shubham"}, "object": {"key": "uploads/49add0d1-b6e2-4dc9-a190-5d838bf34b91/contract.pdf", "size": 875823, "eTag": "deb96679053b2d64e89d0ff5514bb94", "sequencer": "0069EA496129644822"}}]}
2026-04-23T16:31:30.751Z	Started Step Functions execution for job_id=49add0d1-b6e2-4dc9-a190-5d838bf34b91, executionArn=arn:aws:states:eu-west-1:754056706645:execution:ContractReviewPipeline:3311dfac-a6f6-4544-81ae-2dc4ee2bee0f
2026-04-23T16:31:30.753Z	END RequestId: 36e9adf2-4766-4769-9aea-cfb90376cdea
2026-04-23T16:31:30.753Z	REPORT RequestId: 36e9adf2-4766-4769-9aea-cfb90376cdea Duration: 458.34 ms Billed Duration: 459 ms Memory Size: 128 MB Max Memory Used: 95 MB
No newer events at this moment. Auto retrv noused. Resume	

Figure 4: CloudWatch Logs of the pipeline

It updates the DynamoDB status to PROCESSING and starts the ContractReviewPipeline state machine, as observed in Figure 5. I used Step Functions here because Textract is asynchronous. A single long-running Lambda would have been harder to observe and less aligned with AWS best practices.

Table: ContractAnalysisJobs - Items returned (7)						Actions	Create item
Scan started on April 23, 2026, 18:15:17							
☐	job_id (String)	created_at	original_filename	status	upload_s3_key		
☐	test-123			PROCESSING			
☐	49add0d1-b6e2-4dc9...	2026-04-23...	sample-contract.pdf	PROCESSING	uploads/49add0d1-b6e2-4dc9...		
☐	a94455fa-b972-4b5e...	2026-04-23...	contract.pdf	PENDING	uploads/a94455fa-b972-4b5e...		

Figure 5: DynamoDB Table

Inside the workflow, the first major stage is OCR. StartTextract begins the Textract job, WaitForTextract avoids wasteful polling inside Lambda, and CheckTextractResult keeps checking until extraction is complete (Amazon Web Services, 2026). The OCR output is saved to S3 as both

Implementation of an AI-assisted contract review pipeline on Amazon Web Services (AWS)

textract.json and raw_text.txt. That split helped me during testing because the raw text was easier to inspect quickly, while the full Textract JSON preserved the detailed output for debugging.

The next stage is clause preprocessing. ClauseClassifierAndRiskProcessor reads raw_text.txt, splits the content into clause-sized records, adds rough clause labels, and applies deterministic risk hints. I kept this stage rule-based on purpose. It gives me an auditable intermediate file, clauses.json. The deterministic stage is not the final answer but a structured handoff to the semantic AI stage.

For the final AI stage, I used Amazon Bedrock. This project needed semantic interpretation of legal wording, plain-English explanations, and summarisation more than it needed custom model training infrastructure.

The retrieval path is kept separate from the processing path. GET /status/{jobId} returns lightweight state information from DynamoDB, while GET /results/{jobId} returns the final summary, risk counts, metadata, and flagged clauses when the job is complete. I liked this separation because it keeps polling cheap and makes the final result contract easier to manage.

Model Description

I did not train a custom model for this project. Instead, I used a hybrid approach that combines deterministic Python code with managed foundation model inference on AWS.

The following Figure 6 shows my implementation of a State Machine with AWS Step Functions service. There are 2 main types of workflows, Standard and Express, both having their pros and cons (Amazon Web Services, 2026). I went ahead with the Express workflow for its simplicity. The S3 bucket automatically triggers this step function pipeline if it notices that a new file is created under 'uploads/' directory.

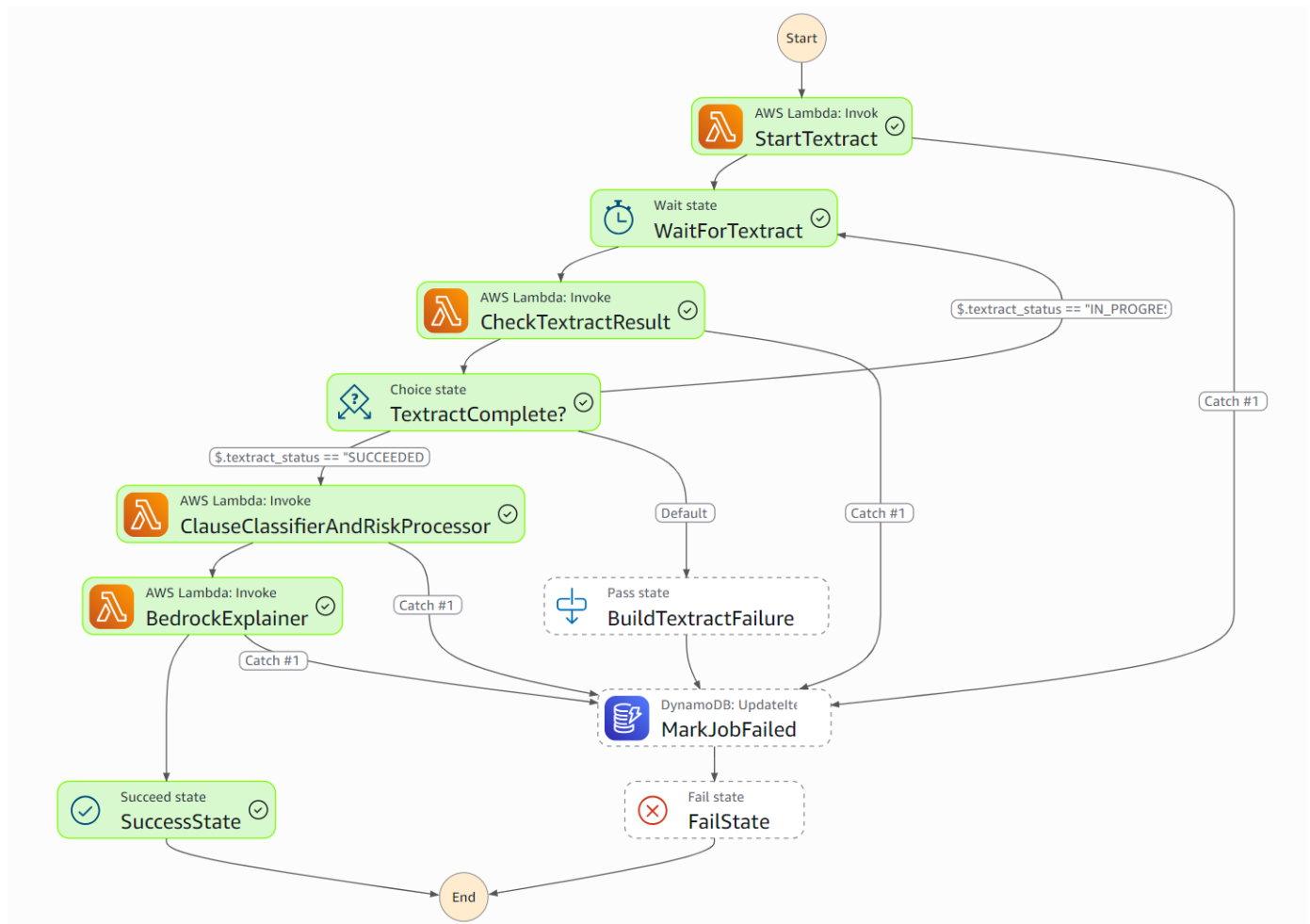


Figure 6: Step Function State Machine implementation graph

The first stage of analysis is deterministic preprocessing in ClauseClassifierAndRiskProcessor. After Textract finishes, this Lambda reads the extracted text, splits it into clauses, assigns rough clause-type hints, and produces candidate severity flags using simple rules. I treated this as transparent preprocessing rather than as the final risk engine. Its main output is clauses.json, which is easier to inspect than raw OCR text and easier for the next stage to consume.

The second stage is semantic analysis in BedrockExplainer. This Lambda loads clauses.json, sends the clause content to Amazon Bedrock, and asks for a final risk assessment, explanation, and contract-level summary. We have an implementation around the Bedrock Converse API with the model identifier currently documented as eu.amazon.nova-pro-v1:0, together with

Implementation of an AI-assisted contract review pipeline on Amazon Web Services (AWS)

bedrock_semantic_analysis, analysis prompts, and summary prompts in the stored metadata (Amazon Web Services, 2026).

The input data for testing came from a dummy contract I created using generative AI, which was exported as a pdf for the upload flow. I designed that fixture to include a mix of intentionally low-risk, medium-risk, and high-risk clauses so the pipeline would have meaningful output to analyse. The high-risk areas are mainly around customer data rights, indemnity, liability, and supplier termination powers. Medium-risk areas include renewal, fee increases, and audit rights. Low-risk examples include governing-law and venue style clauses that are still relevant but less likely to trigger urgent review.

The following Figure 7 shows the result API being called. A large JSON file is created highlighting each clause, as seen in Figure 8, which can be expanded on the frontend. But since this project does not have a frontend, it would not make any sense to output the entire json in the results API. Hence, I am just calling a few basic parameters.

```
PS C:\Users\Shubham> curl.exe -X GET "https://laex943sbc.execute-api.eu-west-1.amazonaws.com/results/4079939e-647e-411a-a85e-26ee102e7260"
{"job_id": "4079939e-647e-411a-a85e-26ee102e7260", "status": "COMPLETE", "summary": "This contract outlines the terms for a software subscription and services agreement between Red Oak Analytics, Inc. and Northwind Retail Ltd., including definitions, services provided, customer obligations, term and renewal, fees and payment, and customer data and security.", "results_s3_key": "processed/4079939e-647e-411a-a85e-26ee102e7260/final_results.json", "high_risk_count": 0, "medium_risk_count": 4}
```

Figure 7: Successful Results API call

The screenshot shows the AWS S3 console interface. The breadcrumb navigation indicates the path: Amazon S3 > Buckets > contract-analyzer-bucket-shubham > processed/. The bucket name '2adb72a4-0399-4c39-b5e4-752c916b5209/' is displayed. The 'Objects' tab is active, showing a list of 4 objects. Above the list are buttons for 'Copy S3 URI', 'Copy URL', 'Download', 'Open', 'Delete', 'Actions', and 'Create folder'. A search bar prompts 'Find objects by prefix'. The table lists the following objects:

Name	Type	Last modified	Size	Storage class
clauses.json	json	April 24, 2026, 15:41:38 (UTC+01:00)	24.8 KB	Standard
final_results.json	json	April 24, 2026, 15:41:43 (UTC+01:00)	13.0 KB	Standard
raw_text.txt	txt	April 24, 2026, 15:41:37 (UTC+01:00)	15.6 KB	Standard
extract.json	json	April 24, 2026, 15:41:37 (UTC+01:00)	1.4 MB	Standard

Figure 8: Files under the /processed/{jobid}/ directory

I deployed the code to the cloud as small AWS Lambda functions connected by Step Functions, rather than as one large application server. That matters for the model discussion because the "model" is not running in isolation. Referring to Figure 8, the text extraction stage writes processed/{job_id}/extract.json and processed/{job_id}/raw_text.txt, the preprocessing stage writes processed/{job_id}/clauses.json, and the Bedrock stage writes processed/{job_id}/final_results.json. DynamoDB stores the corresponding summary fields and metadata so the API can return the results cleanly.

Implementation of an AI-assisted contract review pipeline on Amazon Web Services (AWS)

The rule based classification before the Bedrock step needs further fine tuning or a complete revamp as the output I got was not accurate. I expected at least 1 high_risk_count clause, but I got none. Only medium_risk_count clauses were flagged. Although the classification step needs further work, the entire AWS architecture as a whole performs as intended and is scalable.

Scalability Considerations

This architecture is well suited to scaling because each stage can grow independently instead of forcing the whole system to scale as one unit. If the number of uploaded contracts increased sharply, the first services affected would be API Gateway, Lambda, S3, Step Functions, Textract, Bedrock, and DynamoDB. The main design advantage of my system is that those services are already separated by responsibility, so higher demand in one part of the workflow does not require a complete redesign of the rest of the application. The 7 pillars of well-architected framework include Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization and Sustainability. (Patrick Denny, 2026). By using serverless architecture and dividing the tasks into smaller responsibilities and by having strict IAM Roles and Access policies for the services, I believe that I have achieved all of them.

The upload stage would scale first through the existing presigned-URL design (Amazon Web Services, 2026). Because the PDF is uploaded directly to S3, Lambda does not need to receive or hold the file contents itself. That removes a major bottleneck at ingestion. If hundreds or thousands of users were uploading contracts, API Gateway and UploadHandler would still only be responsible for generating a `job_id`, writing a small DynamoDB item, and returning the upload URL. In other words, the high-volume binary transfer is pushed onto S3, which is the right AWS service for that job.

The processing stage would need the most careful planning as volume grows. The 4 pillars for compute cost optimization are right compute Size, Elasticity, Pricing model, Right Storage type (Patrick Denny, 2026). Since we are using Lambda, the first pillar of compute size is taken care of by AWS. Lambda is elastic by default but has concurrency limit of 1000. If volume grows beyond that, we may need to request additional quota from AWS (Amazon Web Services, 2026). Lambda functions are cost effective as they are not constantly running in the background, and S3 storage is also the right choice for storing objects like pdfs and jsons, ticking off the remaining 2 pillars.

To further discuss the storage scaling, S3 has virtually unlimited storage and it is designed for 11 9s of durability (Patrick Denny, 2026). So, for continuous and uninterrupted storage, S3 is the best for scaling.

Each uploaded file starts a Step Functions execution and then triggers Textract, clause preprocessing, and Bedrock analysis. If demand increased from a few contracts to many concurrent contracts, I would control scaling with queue-like orchestration behaviour at the workflow level rather than trying to make one Lambda do more work. For example, if Textract or Bedrock throughput became the limiting factor, the state machine could be adjusted to introduce rate control, retries, or batching strategies around those specific stages without changing the upload API or storage layout.

Document size also changes the scaling picture. A short agreement and a large multi-page contract do not cost the same amount of OCR time, model tokens, or storage. In this application, larger files create more pages for Textract, more text to reconstruct, more clauses to split, and more tokens to send to Bedrock. I would approach that by keeping the intermediate artifacts already used in this project, such as `raw_text.txt`, `clauses.json`, and `final_results.json`, and then extending the workflow so that very large contracts could be chunked into smaller analysis units before the Bedrock stage. That would let the system scale by clause groups or document sections rather than forcing one model call to handle the entire contract at once.

Implementation of an AI-assisted contract review pipeline on Amazon Web Services (AWS)

I have used 'Express Workflow' while creating Step Functions. But an Express Workflow has a maximum duration of 5 minutes (Amazon Web Services, 2026). This limit is not a concern for most of the cases, but during situations where a big contract is uploaded and the processes like Textraction and Bedrock inferencing is taking time, our workflow could fail. Shifting to 'Standard Workflow' type would be a better solution this edge case.

The data layer is also important for scalability. DynamoDB is a good fit here because each job is naturally keyed by job_id, and the access patterns are simple: create a record, update status fields, and read back status or results. It also has no limits on table size, single digit millisecond latency at any scale and high availability, making it stable for scaling (Patrick Denny, 2026). If the system scaled to a much larger number of jobs, I would keep the core table design but pay more attention to item size, result summaries stored in DynamoDB versus S3, and retention policies for older jobs. S3 already helps with this because the large artifacts live outside the table, so DynamoDB is mainly used for fast metadata lookup rather than document storage.

AI inference is the area where scalability affects both performance and cost most directly. In this system, Bedrock is used only after Textract and rule-based preprocessing have already reduced the problem into clause-level structured input. That is a good scaling decision because it avoids sending raw PDF data or unnecessary text straight to the model. If the application had to support many more contracts, I would optimise this stage by limiting explanation length for low-risk clauses, keeping prompts structured, and only sending the text needed for each clause-level judgment. That would reduce token usage while preserving the quality of the final output.

Another important scalability consideration is observability. When only one contract is being tested manually, it is easy to inspect logs after the fact. At larger scale, that is not enough. In this application, CloudWatch logs, Step Functions execution history, and DynamoDB status fields already provide a strong base for monitoring. If usage increased significantly, I would extend that with clearer operational metrics such as average end-to-end processing time, Textract wait duration, Bedrock latency, clause counts per contract, failure counts by workflow stage, and the number of jobs simultaneously in PENDING, PROCESSING, and COMPLETE.

To conclude, the main reason this application can scale is that the architecture is event-driven and service-oriented. S3 handles file growth, Lambda handles short compute tasks, Step Functions handles orchestration, Textract handles OCR demand, Bedrock handles semantic inference, and DynamoDB handles job tracking. That means scaling the application is less about replacing the architecture and more about tuning concurrency, throughput, chunking strategy, and cost controls at the stages where demand grows fastest.

References

- Amazon Web Services. (2026, April 24). *Amazon API Gateway Documentation*. Retrieved from AWS Documentation: <https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api.html>
- Amazon Web Services. (2026, April 24). *Bedrock Converse API*. Retrieved from AWS Documentation: <https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference-call.html>
- Amazon Web Services. (2026, April 24). *Choosing workflow type in Step Functions*. Retrieved from AWS Documentation: <https://docs.aws.amazon.com/step-functions/latest/dg/choosing-workflow-type.html>
- Amazon Web Services. (2026, April 24). *Detecting Text using Textract*. Retrieved from AWS Documentation: <https://docs.aws.amazon.com/textract/latest/dg/document-response.html>
- Amazon Web Services. (2026, April 24). *Process Amazon S3 event notification with Lambda*. Retrieved from AWS Documentation: <https://docs.aws.amazon.com/lambda/latest/dg/with-s3.html>
- Amazon Web Services. (2026, April 24). *Step Functions Developer Guide*. Retrieved from AWS Documentation: <https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html>
- Amazon Web Services. (2026, April 24). *Understanding Lambda function scaling*. Retrieved from AWS Documentation: <https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html>
- Amazon Web Services. (2026, April 24). *Using Pre-signed URLs*. Retrieved from AWS Documentation: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html>
- Patrick Denny. (2026, February). *Week 4 - Networking, Content Delivery and Compute*. Retrieved from CS5024 - Theory and Practice of Advanced AI Ecosystems: <https://learn.ul.ie/d2l/le/lessons/73419/units/1185773>
- Patrick Denny. (2026, February). *Week 5 - Storage and Databases*. Retrieved from CS5024 - Theory and Practice of Advanced AI Ecosystems: <https://learn.ul.ie/d2l/le/lessons/73419/units/1185774>
- Patrick Denny. (2026, March). *Week 6 - Cloud Architecture*. Retrieved from CS5024 - Theory and Practice of Advanced AI Ecosystems: <https://learn.ul.ie/d2l/le/lessons/73419/units/1185775>